Parte II · Coordinación

Modelos, fallas, tiempo, líderes, replicación y transacciones

Sistemas Distribuidos

Basado en R. Vitillo, *Understanding Distributed Systems*, capítulos 6 a 13

Universidad Industrial de Santander

Agenda

1. **Fundamentos** — modelos de sistema (cap. 6), detección de fallas (cap. 7), tiempo (cap. 8)
2. **Líder y replicación** — elección de líder (cap. 9), replicación y consistencia (cap. 10)
3. **Evitar la coordinación** — difusión, CRDT, quórums, CALM, causalidad (cap. 11)
4. **Transacciones** — ACID, 2PC, NewSQL (cap. 12), *outbox* y sagas (cap. 13)

¿Por qué coordinar?

- Para tener **alta disponibilidad** hay que **replicar**: si un nodo cae, otro responde
- Pero las réplicas deben **ponerse de acuerdo**: qué valor es el actual, quién acepta escrituras, en qué orden
- Y hay que hacerlo con:
  - mensajes que se **pierden** o **retrasan**
  - procesos que **caen** y se recuperan
  - **sin un reloj global** confiable

> La Parte II construye, paso a paso, las herramientas para **cooperar a pesar de las fallas**.

1. Fundamentos

Modelos de sistema · Detección de fallas · Tiempo (caps. 6–8)

Modelos de sistema (cap. 6)

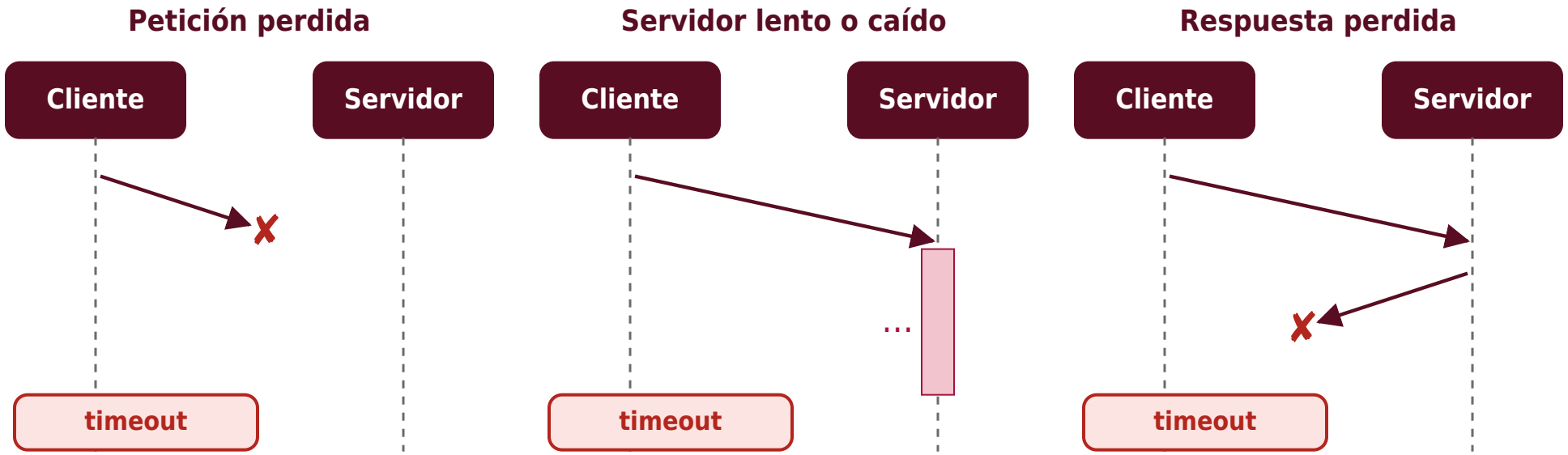
Enlaces	Procesos	Temporización
Pérdida justa	Falla arbitraria (bizantina)	Síncrono
Confiable	**Caída con recuperación**	**Parcialmente síncrono**
Confiable autenticado	Caída con detención	Asíncrono

★ Modelo que asume el libro salvo indicación contraria

¿Por qué importan los modelos?

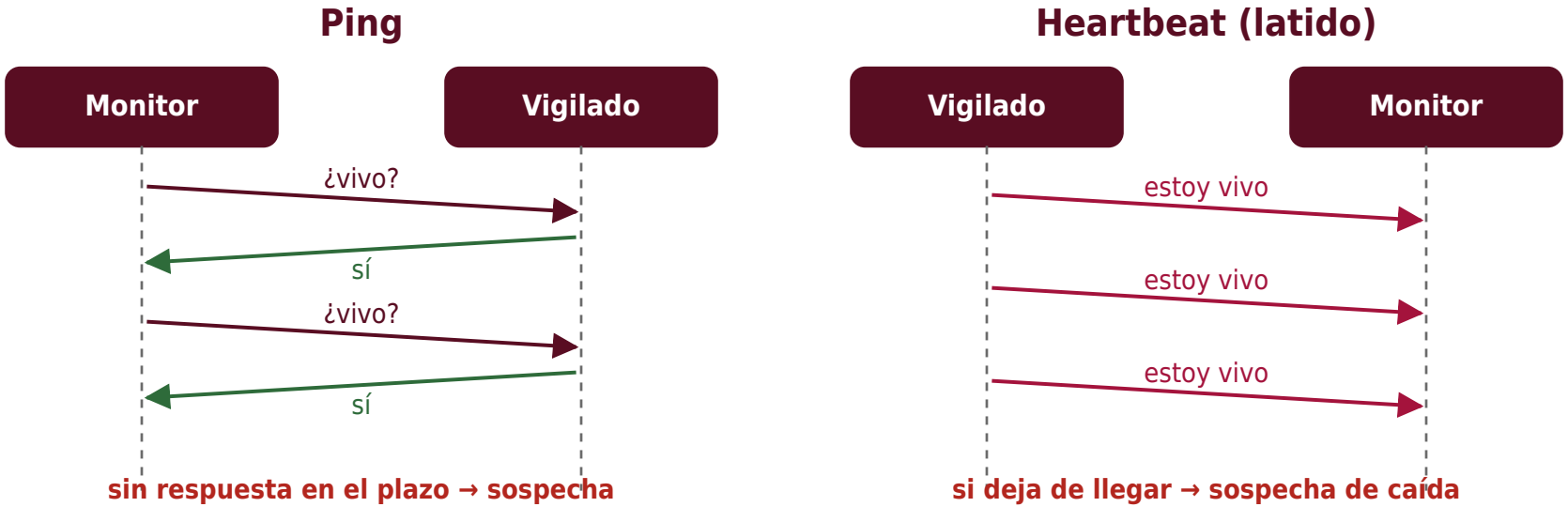
- Un algoritmo es correcto **solo bajo los supuestos** de su modelo
- **Bizantino**: el proceso puede mentir o actuar maliciosamente → algoritmos mucho más costosos (blockchains, aviónica)
- **Caída con recuperación**: pierde lo que estaba en memoria; conserva lo persistido → hay que **persistir** el estado crítico antes de responder
- **Asíncrono puro**: no hay forma de distinguir un proceso lento de uno caído
- **Parcialmente síncrono**: casi siempre hay límites de tiempo… salvo cuando no → los algoritmos deben ser **seguros siempre** y **progresar** cuando el sistema se porta bien

Detección de fallas (cap. 7): el dilema



Para el cliente los tres casos son idénticos: solo puede sospechar

Timeouts, ping y heartbeat



- *Timeout* **corto**: falsos positivos, reintentos de más · **largo**: reacción lenta ante caídas reales
- Un **detector de fallas** solo puede **sospechar**: es imperfecto por naturaleza

Tiempo (cap. 8): relojes físicos

- Cada máquina tiene un reloj de cuarzo que **deriva**
- **NTP** lo sincroniza estimando la latencia de red, pero con **error** y posibles **saltos** (atrás o adelante)
- ⇒ No usar el **reloj de pared** para ordenar eventos entre máquinas ni para medir intervalos
- Para medir duraciones: **reloj monótono** (nunca retrocede)

```
import time
t0 = time.monotonic()          # ✔ medir duraciones
hacer_trabajo()
print(time.monotonic() - t0)
time.time()                    # ✘ puede saltar si NTP ajusta el reloj
```

La relación *sucedió-antes* (→)

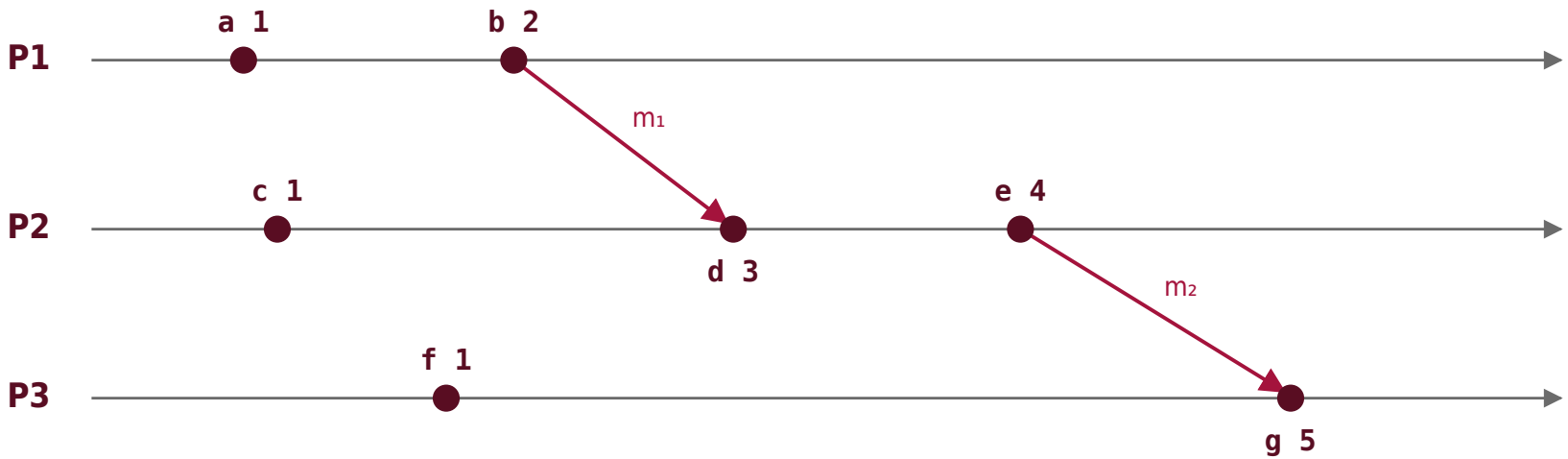
a → b si:

1. a y b ocurren en el **mismo proceso** y a ocurre antes que b, o
2. a es el **envío** de un mensaje y b su **recepción**, o
3. existe c tal que a → c y c → b (**transitividad**)

Si ni a → b ni b → a, los eventos son **concurrentes** (a ∥ b).

> No importa *cuándo* ocurrió algo, sino **qué pudo haber influido en qué**.

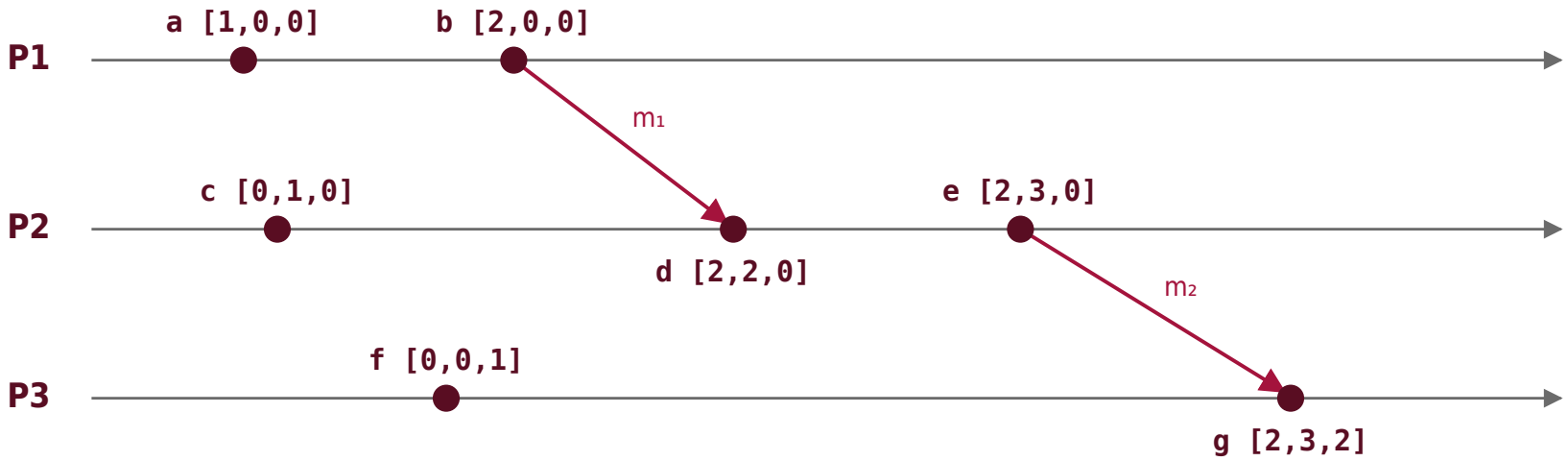
Relojes de Lamport



L(f) = 1 < L(b) = 2, pero f y b son concurrentes: Lamport no detecta concurrencia

- Incrementar antes de cada evento · enviar el valor · al recibir: `max(local, recibido) + 1`
- a → b ⇒ L(a) < L(b), **pero no al revés** · desempate por id de proceso → **orden total**

Relojes vectoriales



f = [0,0,1] y b = [2,0,0]: ninguno es menor que el otro → concurrentes

- Un contador **por proceso** · al recibir: máximo componente a componente y luego incrementar el propio
- V(a) < V(b) **si y solo si** a → b · si ninguno es menor → **concurrentes**

Lamport frente a vectorial

	Lamport	Vectorial
Tamaño	Un entero	Un entero **por proceso**
$a \rightarrow b \Rightarrow$	$L(a) < L(b)$	$V(a) < V(b)$
$L(a) < L(b) \Rightarrow a \rightarrow b$	**No**	Sí ($V(a) < V(b) \Leftrightarrow a \rightarrow b$)
Detecta concurrencia	No	**Sí**
Uso típico	Orden total, marcas de versión	Detección de conflictos entre réplicas

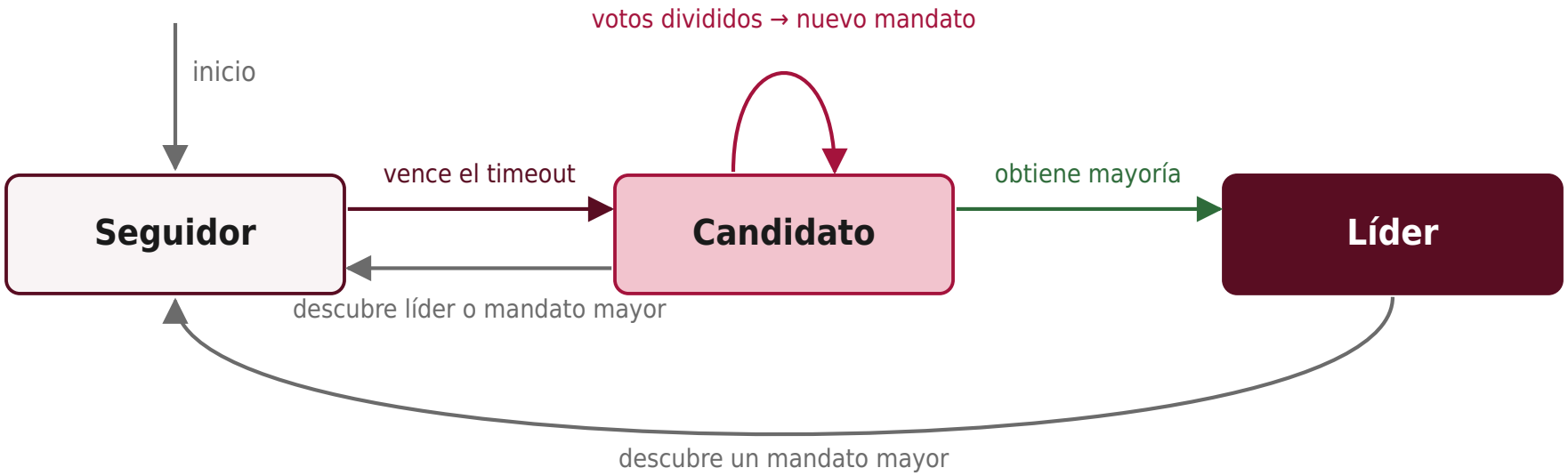
2. Líder y replicación

Elección de líder · Replicación · Consistencia (caps. 9–10)

Elección de líder (cap. 9)

- Muchas tareas necesitan **un solo** coordinador: aceptar escrituras, asignar trabajo, tomar un recurso exclusivo
- Propiedades requeridas:
  - **Seguridad** (*safety*): **como máximo un líder** a la vez — "nunca pasa algo malo"
  - **Vivacidad** (*liveness*): **eventualmente** hay un líder — "algo bueno termina pasando"
- Vitillo usa como ejemplo el algoritmo de elección de **Raft**

Raft: estados y transiciones



Un voto por proceso y por mandato · timeouts aleatorios para evitar empates

Raft: por qué funciona

- **Mandatos** crecientes: cada mensaje lleva el mandato; un mandato mayor "gana"
- **Un voto por mandato** por proceso + **mayoría** ⇒ dos candidatos no pueden ganar el mismo mandato (dos mayorías siempre se solapan)
- **Timeouts aleatorios** (p. ej., 150–300 ms) ⇒ poco probable que varios candidatos empiecen a la vez → evita empates repetidos
- El líder envía ***heartbeats*** para mantener su autoridad

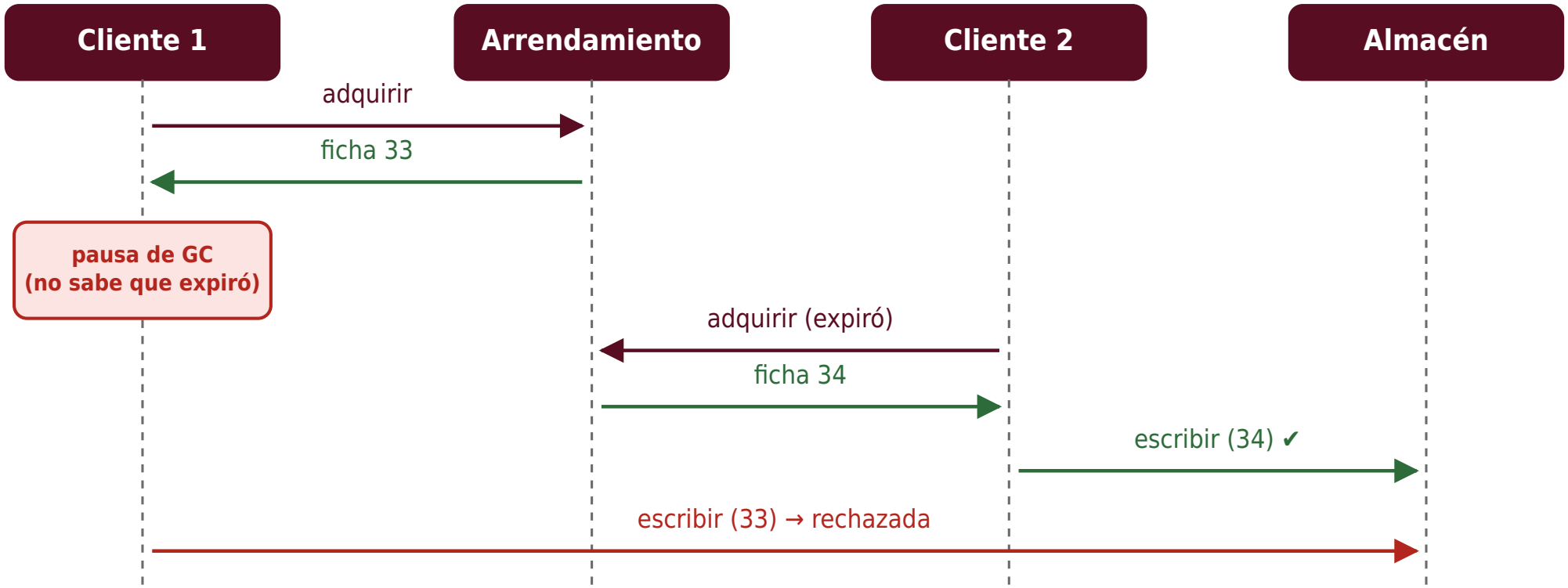
> Seguridad por **mayorías**; vivacidad por **aleatoriedad** y sincronía parcial.

En la práctica: arrendamientos

- Rara vez se implementa la elección desde cero: se usa un **almacén linealizable** (etcd, ZooKeeper, DynamoDB)
- **Arrendamiento** (*lease*): una clave con **TTL**
  - se adquiere con una operación atómica (**compare-and-swap**): "créala solo si no existe"
  - el líder la **renueva** periódicamente; si cae, expira y otro la toma

```
CAS(clave="lider", esperado=None, nuevo="nodo-3", ttl=10s)  → ok
...cada 3 s: renovar(clave="lider", dueño="nodo-3")
```

Fichas de cercado (*fencing tokens*)



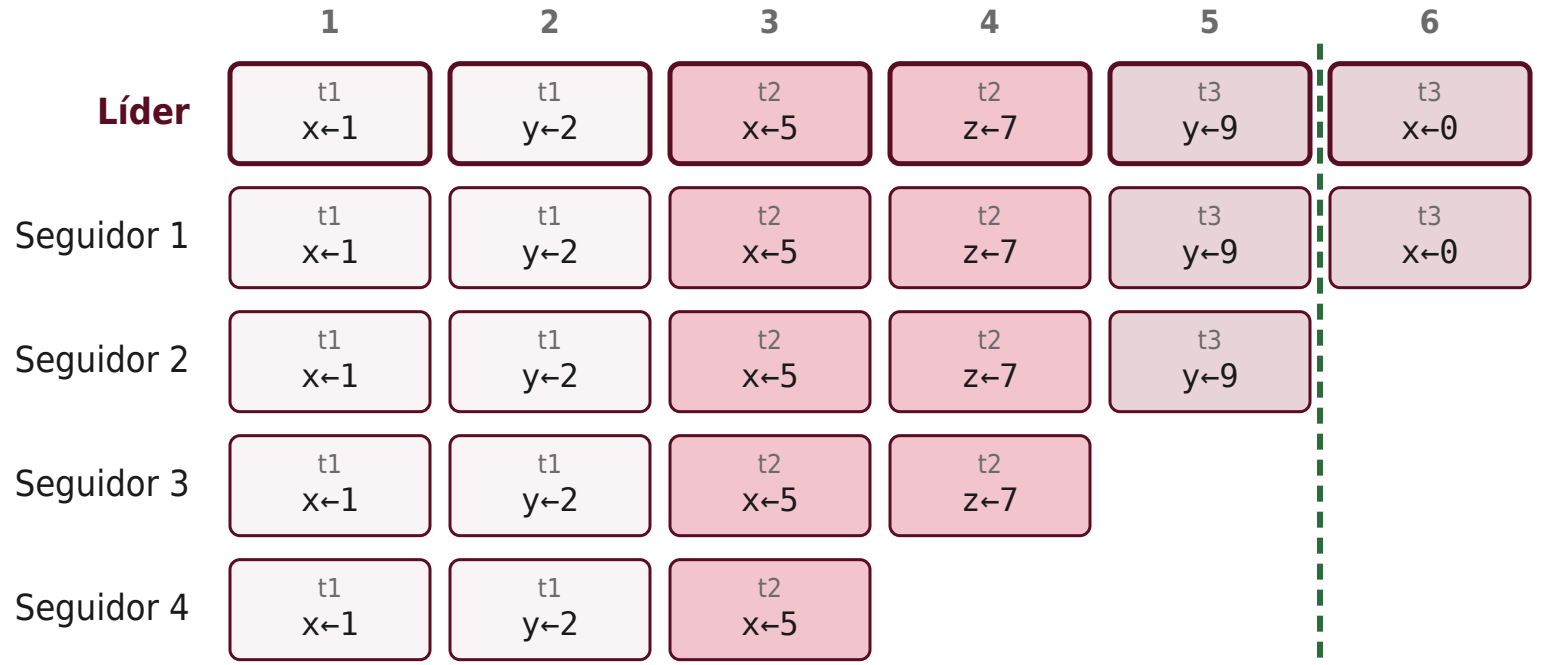
El líder: ¿solución o problema?

- Es un **punto único de falla**: mientras se elige otro, el sistema no avanza
- Es un **cuello de botella**: todo pasa por él
- Recomendaciones del libro:
  - que el líder haga el **menor trabajo** posible
  - usar **fichas de cercado** si el recurso compartido lo permite
  - si se puede, **evitar** necesitar un líder (cap. 11)

Replicación de máquinas de estado (cap. 10)

- Si todas las réplicas parten del **mismo estado** y aplican las **mismas operaciones en el mismo orden**, llegan al **mismo estado**
- El problema se reduce a mantener un **registro** (*log*) idéntico → **consenso**
- Raft:
  i. El líder agrega la operación a su registro
  ii. La envía a los seguidores (`AppendEntries`)
  iii. Cuando la **mayoría** la tiene, la **confirma** y la aplica
  iv. Responde al cliente; los seguidores aplican en el siguiente mensaje

El registro replicado de Raft



La entrada 5 está en 3 de 5 réplicas (mayoría) → confirmada; la 6 aún no

índice de confirmación = 5

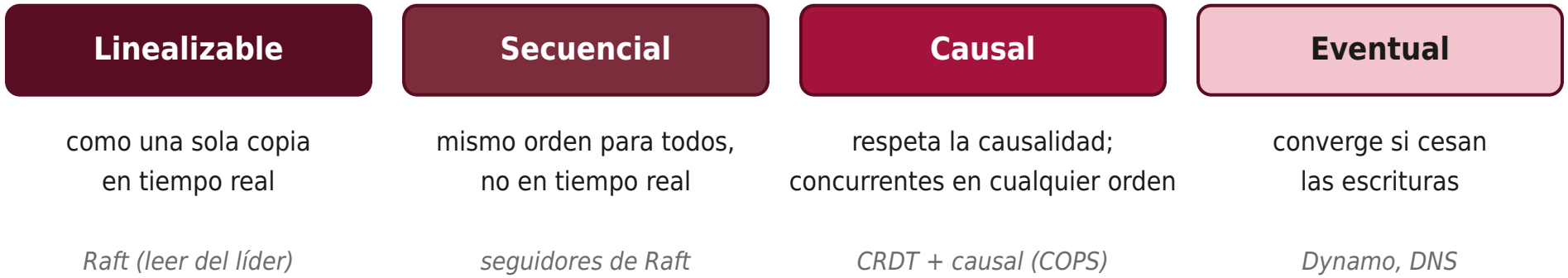
Tolerancia a fallas: 2f + 1

Réplicas (N)	Mayoría	Fallas toleradas (f)
1	1	0
3	2	1
4	3	1
5	3	2
7	4	3

- Con **números pares** no se gana tolerancia: 4 réplicas toleran lo mismo que 3
- Más réplicas ⇒ más tolerancia, pero **más latencia** de escritura

Modelos de consistencia

más fuerte: más coordinación, más latencia → **más débil: más disponibilidad, menos latencia**



Ejemplos de cada modelo

Linealizable (fuerte)

- Una vez que una escritura responde, **toda** lectura posterior ve ese valor o uno más nuevo
- Raft: leer del líder, que **confirma** su liderazgo (o usa un arrendamiento)

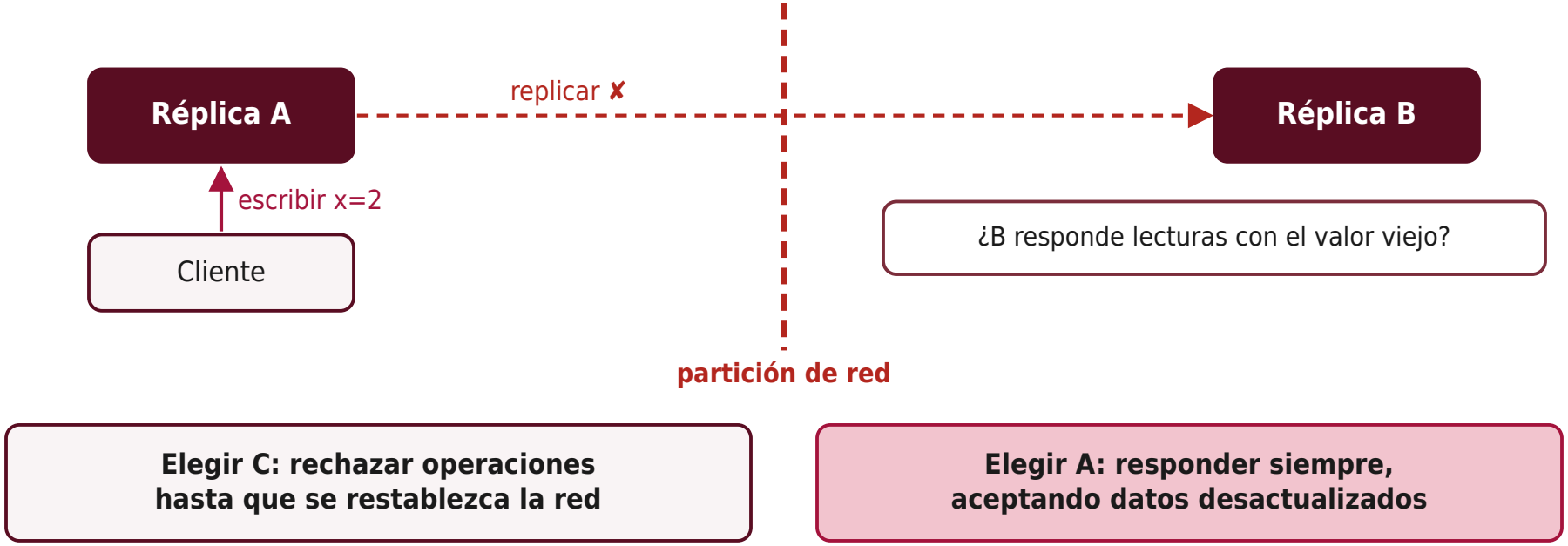
Secuencial

- Todos ven el **mismo orden**, pero un lector puede ir **atrasado**: leer de un seguidor

Eventual

- Se puede leer un valor viejo; si cesan las escrituras, las réplicas **convergen**
- Fácil de implementar, difícil de programar encima: ¿qué pasa si el usuario no ve su propia escritura?

Teorema CAP



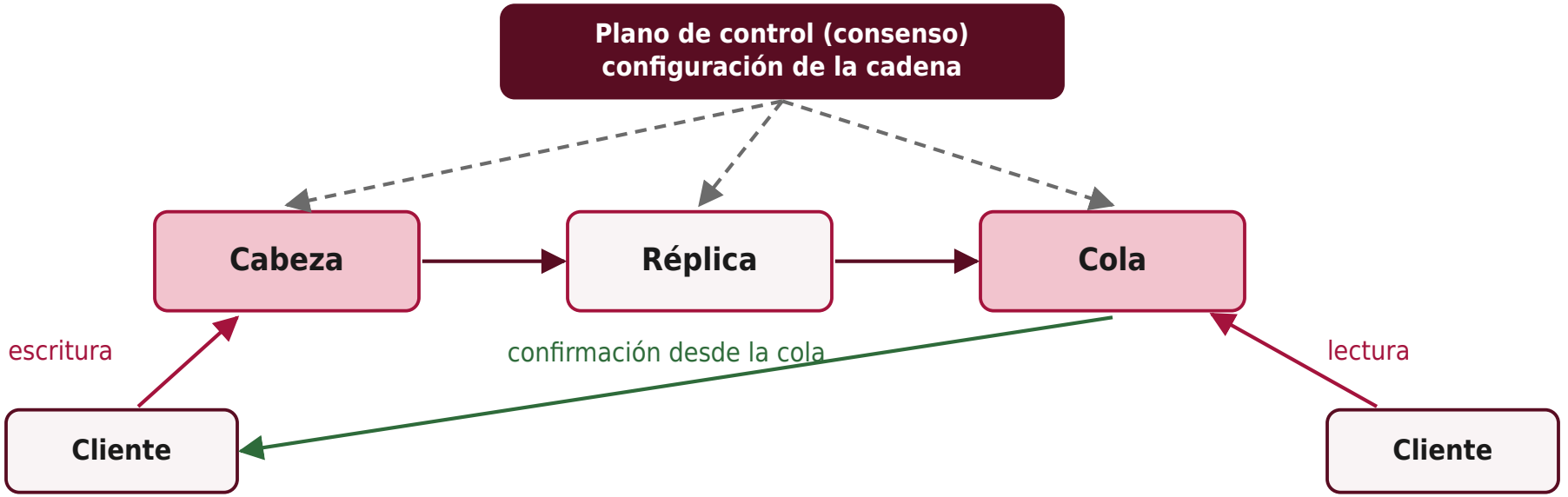
PACELC: si no hay partición, el compromiso es entre latencia y consistencia

PACELC: el compromiso cotidiano

- CAP solo habla del caso de **partición**, que es **raro**
- PACELC: si hay **P**artición → **A** o **C**; **E**n otro caso → **L**atencia o **C**onsistencia
- Consistencia fuerte exige **coordinación** ⇒ más mensajes ⇒ más latencia, **siempre**

Sistema (ejemplos)	Partición	Normal
etcd, ZooKeeper, Spanner	C	C
Dynamo / Cassandra (configuración típica)	A	L
Bases de datos con réplicas de lectura asíncronas	A (lecturas)	L

Replicación en cadena



Escrituras entran por la cabeza; lecturas en la cola → consistencia fuerte sin consenso en el plano de datos

Cadena frente a Raft

	Raft	Replicación en cadena
Escrituras	Al líder; confirma con mayoría	Por la cabeza; confirma la cola
Lecturas fuertes	Del líder	De la cola (CRAQ: de cualquiera, consultando a la cola si hay escrituras pendientes)
Latencia de escritura	1 ida y vuelta a la mayoría	Recorre **toda** la cadena
Nodo lento	La mayoría lo enmascara	**Frena** toda la cadena
Consenso	En el camino de datos	Solo en el **plano de control**

3. Evitar la coordinación

Difusión · CRDT · Quórums · CALM · Causalidad (cap. 11)

Protocolos de difusión (*broadcast*)

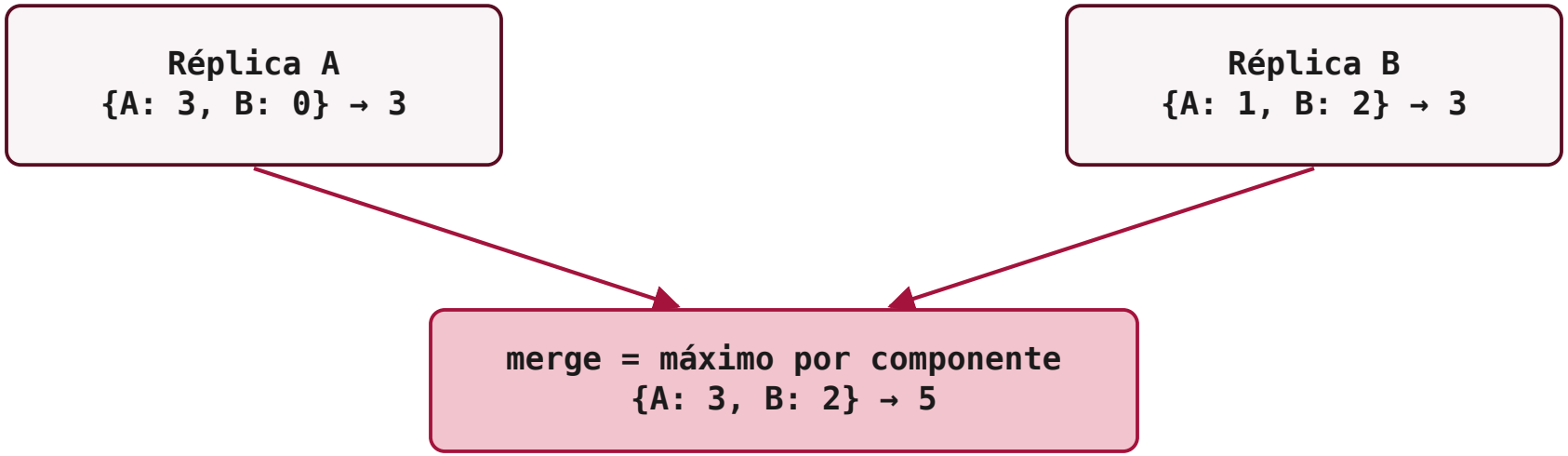
Tipo	Garantía	Costo
Best-effort	Si el emisor no cae, todos reciben	Bajo; sin garantía si el emisor cae a mitad
Confiable (ansiosa)	Si un proceso correcto entrega, **todos** entregan	Cada proceso reenvía a todos: $O(N^2)$ mensajes
Chismes (*gossip*)	Confiable con alta probabilidad	Cada uno reenvía a unos pocos al azar
Orden total	Todos entregan en el **mismo orden**	Requiere **consenso**

> Difusión de orden total ≡ consenso ≡ replicación de máquinas de estado.

CRDT: datos que convergen solos

- Réplicas que aceptan escrituras **localmente** y se envían su estado
- Se combinan con una operación **merge** que es:
  - **conmutativa**: merge(a, b) = merge(b, a) → no importa el orden
  - **asociativa**: merge(merge(a, b), c) = merge(a, merge(b, c)) → no importa el agrupamiento
  - **idempotente**: merge(a, a) = a → no importan los duplicados
- ⇒ **Consistencia eventual fuerte**: réplicas que recibieron las mismas actualizaciones están en el **mismo estado**

Ejemplo: contador creciente (G-Counter)



Conmutativa · asociativa · idempotente → converge en cualquier orden y con duplicados

Otros CRDT

- **Registro "último escritor gana"** (LWW): cada valor lleva una marca de tiempo (p. ej., Lamport); el merge conserva la mayor
  - simple, pero **descarta** escrituras concurrentes
- **Registro multivalor** (MV): conserva **todas** las escrituras concurrentes (detectadas con relojes vectoriales) y deja que el cliente resuelva
- **Contadores con decremento** (PN-Counter): dos G-Counter, uno para sumas y otro para restas
- **Conjuntos**: con marcas para distinguir agregar y quitar (p. ej., OR-Set)

> Se pueden **componer**: un mapa de registros, un documento colaborativo...

Almacenes estilo Dynamo: quórums

N = 3 réplicas · W + R = 4 > N



Todo quórum de lectura se solapa con todo quórum de escritura (aquí, en B)

Convergencia: reparación en lectura + antientropía (árboles de Merkle)

Convergencia sin líder

- **Reparación en lectura** (*read repair*): si una lectura detecta réplicas con versiones viejas, les envía la nueva
- **Antientropía**: proceso de fondo que compara réplicas y sincroniza las diferencias
  - **árboles de Merkle**: hashes jerárquicos para encontrar qué rangos difieren **sin transferir todo**
- Aun con W + R > N, pueden ocurrir **anomalías** (p. ej., escrituras concurrentes, fallas a mitad de escritura): no es linealizable en general

Teorema CALM y consistencia causal

CALM (*Consistency As Logical Monotonicity*)

- Un problema tiene solución consistente **sin coordinación** ⇔ es **monótono**
- Monótono: agregar información nunca obliga a **retractar** una conclusión
  - ✔ "¿existe un camino entre A y B?" (más aristas no lo invalidan)
  - ✘ "¿*no* existe un camino?" (una arista nueva puede cambiar la respuesta)

Consistencia causal

- Lo causalmente relacionado se ve **en el mismo orden** en todas partes; lo concurrente, en cualquier orden
- Es el modelo **más fuerte** compatible con **disponibilidad** ante particiones (ej.: COPS)

4. Transacciones

ACID · Aislamiento · 2PC · *Outbox* · Sagas (caps. 12–13)

ACID (cap. 12)

Propiedad	Significado
Atomicidad	Todas las operaciones se aplican o ninguna
Consistencia	Se preservan las invariantes de la aplicación (p. ej., saldo ≥ 0)
Aislamiento	Transacciones concurrentes no ven estados intermedios ajenos
Durabilidad	Lo confirmado sobrevive a fallas (persistido / replicado)

> La "C" de ACID no es la "C" de CAP: aquí es responsabilidad compartida con la **aplicación**.

Aislamiento: anomalías y niveles

Anomalía	Ejemplo
Escritura sucia	T_2 sobrescribe un valor que T_1 escribió y no ha confirmado
Lectura sucia	T_2 lee un valor de T_1 que luego se aborta
Lectura no repetible	T_1 lee x dos veces y obtiene valores distintos
Lectura fantasma	Una consulta por rango devuelve filas nuevas al repetirse

Nivel	Evita
Lectura no confirmada	escrituras sucias
Lectura confirmada	+ lecturas sucias
Lectura repetible	+ lecturas no repetibles
Serializable	+ fantasmas: equivale a **alguna** ejecución en serie
Serializabilidad estricta	serializable **+** linealizable (respeta el tiempo real)

Control de concurrencia

Pesimista: bloqueo en dos fases (2PL)

- **Expansión**: se adquieren bloqueos (lectura compartidos, escritura exclusivos) · **Contracción**: se liberan
- Garantiza serializabilidad · riesgo de **interbloqueos** → detección y aborto

Optimista (OCC)

- Ejecutar sin bloquear sobre una copia; al confirmar, **validar** que nadie modificó lo leído; si sí, **reintentar**
- Conviene con **pocos conflictos**

Multiversión (MVCC)

- Se guardan varias versiones; las lecturas usan una **instantánea** consistente y **no bloquean** a las escrituras (*snapshot isolation*)

Atomicidad distribuida: 2PC



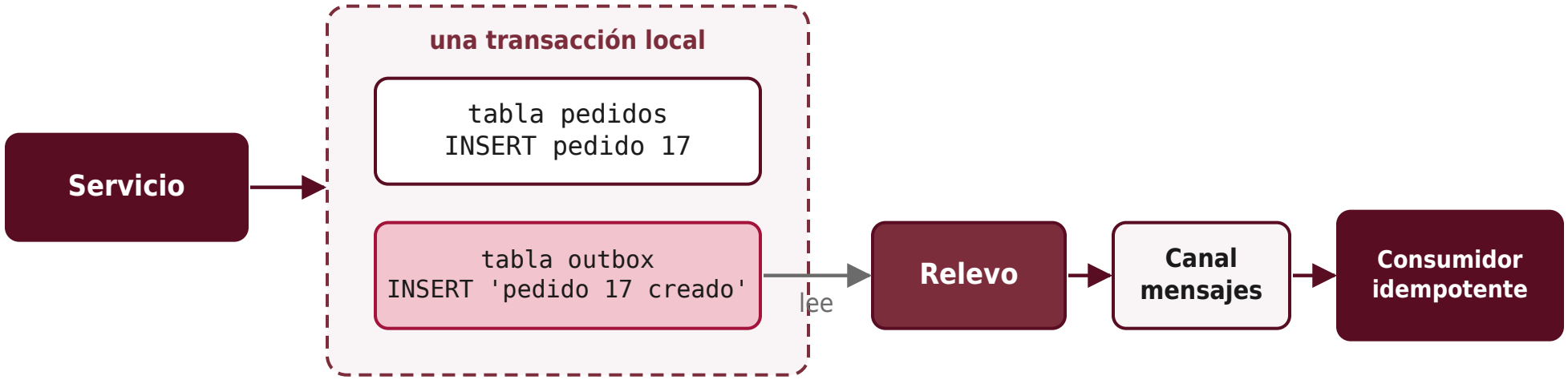
Problemas de 2PC y NewSQL

- **Bloqueante**: si el coordinador cae tras la fase 1, los participantes esperan con **bloqueos tomados**
- **Lento**: varias idas y vueltas y escrituras a disco
- Requiere que **todos** estén disponibles: la disponibilidad empeora con cada participante

NewSQL: Spanner

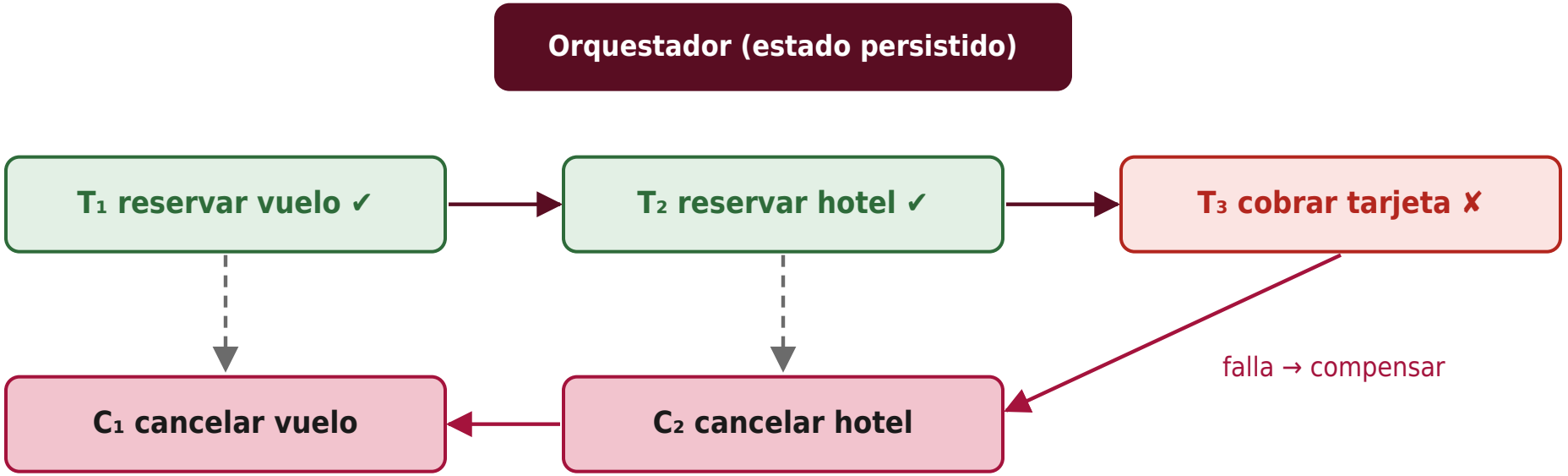
- Cada partición se **replica con Paxos** → participantes y coordinador son **tolerantes a fallas**
- 2PC entre particiones + **MVCC**
- **TrueTime**: relojes (GPS y atómicos) con **incertidumbre acotada** → marcas de tiempo globales → **serializabilidad estricta**

Transacciones asíncronas (cap. 13): *outbox*



Cambio y mensaje se guardan juntos o no se guardan · entrega al menos una vez

Sagas



Transacciones locales + compensaciones idempotentes · sin aislamiento

Sagas en la práctica

- Un **orquestador** ejecuta los pasos y **persiste** el estado de la saga → puede continuar tras una caída
- Comunicación por **mensajes** (al menos una entrega) ⇒ pasos y compensaciones **idempotentes**
- **Sin aislamiento**: otros pueden ver estados intermedios (vuelo reservado sin hotel)
  - mitigación: **bloqueos semánticos** (marcar "pendiente"), diseñar operaciones conmutativas
- Herramientas de flujo de trabajo (p. ej., AWS Step Functions, Temporal) implementan el orquestador

> 2PC: atomicidad + aislamiento, pero bloqueante · Saga: disponible y desacoplada, pero **sin aislamiento**

Ideas clave de la Parte II

1. Los algoritmos son correctos **bajo un modelo**: pérdida justa, caída con recuperación, sincronía parcial
2. Las fallas solo se pueden **sospechar** con *timeouts*
3. Sin reloj global: **Lamport** da orden total; **vectoriales** detectan concurrencia
4. **Raft**: mandatos + mayorías para un líder seguro; **fichas de cercado** contra líderes obsoletos
5. **Replicación**: consenso para consistencia fuerte; **CAP/PACELC** marcan el costo
6. **Sin coordinación**: CRDT, quórums, CALM, consistencia causal
7. **Transacciones**: 2PL/OCC/MVCC, **2PC** (bloqueante), Spanner; asíncronas con **outbox** y **sagas**

Lecturas y referencias

- R. Vitillo, *Understanding Distributed Systems*, 2.ª ed., caps. 6–13
- L. Lamport, "Time, Clocks, and the Ordering of Events in a Distributed System", 1978
- D. Ongaro y J. Ousterhout, "In Search of an Understandable Consensus Algorithm (Raft)", 2014 — visualización: raft.github.io
- G. DeCandia *et al.*, "Dynamo: Amazon's Highly Available Key-value Store", 2007
- J. Corbett *et al.*, "Spanner: Google's Globally-Distributed Database", 2012
- H. Garcia-Molina y K. Salem, "Sagas", 1987

Siguiente: Parte III — Escalabilidad

¿Preguntas?

Parte II · Coordinación

Sistemas Distribuidos
Universidad Industrial de Santander